

Topic 8: Backend Architecture — FastAPI, Threadpools, & DoS Guards

TOPIC OBJECTIVE

Serving clinical decision support systems requires balancing ultra-responsive web I/O with computationally intensive graph execution. In api.py, FastAPI leverages Starlette's asynchronous event loop and automatic worker threadpools to process clinical requests without blocking concurrency, while enforcing strict 5MB payload caps and input sanitization to eliminate Denial of Service (DoS) attack vectors.

1. The Real-World Analogy: The Maître D' & The Kitchen Brigade

Imagine a bustling, Michelin-starred restaurant with hundreds of patrons arriving simultaneously:

- **The Maître D' (FastAPI Async Event Loop):** Stands at the front desk. Welcomes guests, verifies reservations, hands out menus, and distributes bills. The Maître D' handles thousands of interactions per hour because they never step into the kitchen to cook.
- **The Kitchen Brigade (Starlette Worker Threadpool via def endpoints):** When a complex meal is ordered (parsing a 400-resource FHIR bundle and executing LangGraph), the Maître D' slips the ticket to the kitchen brigade. Dedicated chefs work on the ticket off to the side without blocking the front door.
- **The Door Guard (DoS Mitigation & 5MB Payload Cap):** If an unruly visitor tries to haul a 100-ton shipping container of raw uninspected food into the dining room (a multi-megabyte FHIR JSON bomb), the door guard stops them instantly with HTTP 413 (Payload Too Large) before kitchen resources are consumed.

2. Concurrency Architecture: The 'def' vs. 'async def' Dilemma

One of the most dangerous anti-patterns in Python asynchronous programming is running CPU-bound or synchronous code inside an `async def` route handler. If you execute `clinical_graph.invoke()` inside an `async def` function, Python's single-threaded event loop freezes completely until the graph finishes, starving all other incoming HTTP requests.

In `api.py`, AegisClinical declares compute-heavy endpoints using standard synchronous `def`:

```
@app.post("/api/evaluate")
def evaluate_custom_bundle(bundle: Dict[str, Any]):
    # FastAPI detects 'def' and automatically offloads execution to Starlette's threadpool
    parsed_data = parse_synthea_bundle(bundle)
    return _evaluate_patient_data(parsed_data)
```

Route Definition	Execution Mechanism	Clinical Concurrency Impact
------------------	---------------------	-----------------------------

async def with CPU-bound tasks	Runs directly on the primary asyncio event loop.	CRITICAL ANTI-PATTERN: Stalls entire server. Other clinicians experience UI freezes & timeouts.
Standard def (AegisClinical Pattern)	Offloaded to Starlette worker threadpool (anyio.to_thread).	OPTIMAL: Event loop remains 100% non-blocking; concurrent evaluations process across CPU cores.

3. Clinical DoS Mitigation: The 5MB Payload Guard

Healthcare EHR exports can easily reach tens of megabytes of nested JSON. Without strict boundaries, unauthenticated users or compromised endpoints could transmit massive JSON documents, triggering Out-Of-Memory (OOM) killer terminations and service crashes.

In api.py (line 130), AegisClinical guards the multipart file upload route with an immediate byte-size check:

```
@app.post("/api/upload")
def upload_custom_bundle(file: UploadFile = File(...)):
    # Enforce strict 5MB ceiling before memory buffer allocation
    if file.size and file.size > 5_242_880:
        raise HTTPException(status_code=413, detail="File exceeds 5MB limit")
    contents = file.file.read()
    bundle = json.loads(contents.decode("utf-8"))
    return _evaluate_patient_data(parse_synthea_bundle(bundle))
```

4. End-to-End Evaluation Request Pipeline

Every evaluation follows a deterministic, memory-safe data transformation pipeline:

Step & Protocol	Component	Description & Guarantees
1. Ingress & CORS HTTP POST	FastAPI Router (CORSMiddleware)	Validates origin headers, inspects Content-Length, routes to worker thread.
2. Parsing In-Memory	FHIR Synthea Parser (src/parser.py)	Extracts creatinine trajectory, medications, and conditions into structured Python dict.
3. Graph Exec Threadpool	LangGraph Engine (src/graph.py)	Executes scatter-gather evaluation across deterministic rules and local SLM agents.
4. Serialization HTTP 200 OK	Pydantic Exporter (.model_dump())	Converts strongly-typed domain models into sanitized JSON response payload.

KNOWLEDGE CHECK & NEXT STEP

1. Why does AegisClinical use standard def instead of async def for evaluation routes?

Because LangGraph execution and FHIR parsing are synchronous CPU-bound operations. Using def instructs Starlette to execute them in background worker threadpools, keeping the main asyncio loop responsive.

2. How does the API prevent Denial-of-Service attacks from oversized FHIR bundles?

By enforcing an instantaneous 5MB (5,242,880 bytes) size check on file uploads, rejecting oversized payloads with HTTP 413 before memory buffers are consumed.

3. Next Step in Topic 9: The Clinical Console — High-Density UI, Skeletons, and Race Conditions (React 19, CSS Tokens).